

UNIVERSIDADE DE MOGI DAS CRUZES
BACHARELADO EM SISTEMAS DE INFORMAÇÃO

FELIPE COSTA MONTEIRO

SISTEMA DE LOGIN SEGURO:
documentação técnica e de requisitos

FELIPE COSTA MONTEIRO

SISTEMA DE LOGIN SEGURO:

documentação técnica e de requisitos

Atividade apresentada à disciplina de Aplicativos Web, do curso de Bacharelado em Sistemas de Informação da Universidade de Mogi das Cruzes, para obtenção da nota M1.

Professor: Prof. Alessandro Aparecido da Silva.

MOGI DAS CRUZES

2026

RESUMO

Este trabalho documenta o Sistema de Login Seguro, que desenvolvi individualmente como atividade para a nota M1 da disciplina de Aplicativos Web, no Bacharelado em Sistemas de Informação da Universidade de Mogi das Cruzes. A aplicação foi feita com Java 21, Spring Boot 4.1.1, Spring Security, Thymeleaf e MongoDB Atlas. Ela permite cadastrar usuários, entrar e sair do sistema e controla o acesso às páginas por três perfis: USER, MANAGER e ADMIN. As senhas são guardadas como hash BCrypt, os dados passam por validação no servidor e as sessões ficam salvas no Atlas, junto com os usuários. O administrador consegue listar, editar e excluir contas. O documento explica a organização do código em camadas, a integração com o banco e o caminho para adaptar o sistema ao Projeto Final de Curso trocando o nome, o tema e os fragmentos da interface. Também registra os requisitos e as regras de negócio com a situação real de cada um e as limitações desta versão.

Palavras-chave: autenticação; autorização; Spring Security; MongoDB Atlas; Thymeleaf.

SUMÁRIO

1 INTRODUÇÃO	4
1.1 PROPÓSITO	4
1.2 ESCOPO	4
1.3 CRITÉRIOS DE REDAÇÃO E CONVENÇÕES	5
2 ESTRUTURA DO SISTEMA	6
2.1 PACOTES E RESPONSABILIDADES	6
2.2 CAMINHO DE UMA OPERAÇÃO	7
2.3 DTOS E MAPPER.....	7
3 INTEGRAÇÃO COM O MONGODB ATLAS	9
3.1 CONFIGURAÇÃO E VARIÁVEL MONGODB_URI	9
3.2 BANCO, COLEÇÕES E ÍNDICE	10
3.3 CONEXÃO SEGURA, USUÁRIO DO BANCO E LIBERAÇÃO DO IP	10
3.4 PRIMEIRO ADMINISTRADOR.....	10
4 PERSONALIZAÇÃO E ADAPTAÇÃO AO PFC	12
4.1 NOME DO SISTEMA E TEMA	12
4.2 CRIAR UM NOVO TEMA	12
4.3 FRAGMENTOS COMPARTILHADOS.....	13
4.4 NOVAS FUNCIONALIDADES E REGRAS DE ACESSO.....	13
5 REQUISITOS E REGRAS DE NEGÓCIO.....	14
5.1 ATORES E PERMISSÕES.....	14
5.2 REQUISITOS FUNCIONAIS	15
5.3 REQUISITOS NÃO FUNCIONAIS	16
5.4 REGRAS DE NEGÓCIO	17
6 CONSIDERAÇÕES FINAIS	19
REFERÊNCIAS.....	20

1 INTRODUÇÃO

A proposta da atividade M1 foi construir, individualmente, um sistema de login seguro com Java Spring Boot, Thymeleaf e MongoDB Atlas, capaz de controlar os usuários de uma aplicação web com autenticação e autorização. Além de funcionar, o sistema precisava servir de base para o Projeto Final de Curso (PFC). Por isso, desde o começo procurei separar a parte de segurança da parte visual, para que a aparência e as funcionalidades possam mudar sem reescrever o núcleo.

No meu projeto, o código está no GitHub, no repositório login-seguro, e foi desenvolvido com o fluxo GitFlow: cada funcionalidade nasceu em uma branch feature, como feature/cadastro, feature/login e feature/sesoes-mongodb, e depois foi integrada à develop.

1.1 PROPÓSITO

Este documento explica como o sistema foi implementado: a organização do código, a integração com o MongoDB Atlas e a forma de adaptar o projeto. Ele também registra os requisitos funcionais, os requisitos não funcionais e as regras de negócio com a situação de cada um no código entregue. Segundo Sommerville (2011, p. 63), o documento de requisitos deve comunicar as necessidades do sistema aos interessados e dar informações precisas a quem desenvolve e testa.

1.2 ESCOPO

Esta versão cobre o cadastro de usuários, o login, o logout, o controle de acesso por perfil, a página inicial do usuário autenticado e a administração de usuários, com listagem, edição de nome, e-mail e perfil e exclusão. Também inclui a personalização do nome do sistema e do tema visual pela configuração.

Ficaram fora desta versão a recuperação de senha por e-mail, a autenticação em duas etapas, o login por redes sociais, uma página completa de perfil do usuário, a busca e a paginação na lista de usuários e a desativação de contas sem exclusão. Esses pontos voltam como melhorias futuras nas considerações finais.

1.3 CRITÉRIOS DE REDAÇÃO E CONVENÇÕES

Os requisitos funcionais começam com “O sistema deverá” e usam verbos que dá para verificar, como cadastrar, autenticar, encerrar, listar, editar e excluir, seguindo o guia de apoio que usamos em aula (Silva, 2026). Esse cuidado segue a recomendação de Sommerville (2011, p. 67-68) de padronizar a forma de escrever os requisitos. A seção 5 apresenta os requisitos e as regras de negócio, e o quadro 1 explica os identificadores e as situações usadas.

Quadro 1 – Identificadores e situações usados no documento

Identificador	Significado
RF01 a RF09	Requisito funcional.
RNF01 a RNF16	Requisito não funcional.
RN01 a RN17	Regra de negócio.
Atendido	Implementado e funcionando na versão entregue.
Parcial	Implementado em parte. A diferença aparece junto à situação.
Não verificado	Sem teste realizado nesta versão.

Fonte: Elaborado pelo autor (2026).

2 ESTRUTURA DO SISTEMA

Como aprendemos em aula, dividir o código em camadas deixa cada classe com uma responsabilidade e facilita a manutenção. No meu projeto, todo o código Java fica no pacote `br.com.felipe.login_seguro`, organizado conforme o quadro 2.

2.1 PACOTES E RESPONSABILIDADES

Quadro 2 – Pacotes do sistema

Pacote	Responsabilidade	Principais classes
config	Configurações de segurança, criptografia de senha, sessões e tema.	SecurityConfig, SenhaConfig, SessaoConfig, TemaConfig
controller	Recebe as requisições, chama o serviço e escolhe a página exibida. Não contém regra de negócio.	UsuarioController, AutenticacaoController, AreaController, AdminUsuarioController, TemaControllerAdvice
dto	Dados que entram pelos formulários e que saem para as páginas, com as anotações de validação.	UsuarioRequestDTO, UsuarioAtualizacaoRequestDTO, UsuarioResponseDTO
entity	Documento gravado no banco e perfis de acesso.	UsuarioEntity, PerfilEnum
mapper	Conversão entre entidade e DTO.	UsuarioMapper
repository	Acesso ao MongoDB pelo Spring Data.	IUsuarioRepository
service	Contrato dos serviços.	IUsuarioService
service.impl	Regras de negócio e ligação com o Spring Security.	UsuarioService, UsuarioDetailsService

Fonte: Elaborado pelo autor (2026).

A interface fica em `src/main/resources`. A pasta `templates` guarda as páginas Thymeleaf, e o arquivo `templates/fragmentos/layout.html` reúne o que todas as páginas compartilham: o head, o cabeçalho com o menu e o rodapé. Os estilos ficam em `static/css`. O arquivo `estilo.css` define o layout, e a pasta `static/css/temas` tem um arquivo CSS por tema, cada um só com as variáveis de cor.

2.2 CAMINHO DE UMA OPERAÇÃO

O quadro 3 acompanha um cadastro desde o formulário até o banco, passando por formulário, controller, service, repository e MongoDB. As outras operações seguem o mesmo caminho.

Quadro 3 – Caminho de um cadastro pelas camadas

Etapa	Camada	O que acontece
1	Formulário (cadastro.html)	O visitante preenche nome, e-mail e senha. O Thymeleaf inclui o token CSRF no formulário, e o navegador envia um POST para /cadastro.
2	Controller (UsuarioController)	O Spring monta um UsuarioRequestDTO e valida as anotações @NotBlank, @Email e @Size. Se houver erro, a página volta com a mensagem junto ao campo.
3	Service (UsuarioService)	Normaliza o e-mail (sem espaços nas pontas e em minúsculas), recusa e-mail já cadastrado, confere o limite de 72 bytes da senha, define o perfil USER e gera o hash BCrypt.
4	Mapper (UsuarioMapper)	Converte o DTO em UsuarioEntity e, depois de salvar, converte a entidade em UsuarioResponseDTO.
5	Repository (IUsuarioRepository)	O método save do Spring Data grava o documento.
6	MongoDB Atlas	O documento entra na coleção usuarios. O índice único do e-mail barra a duplicidade mesmo se duas requisições chegarem ao mesmo tempo.
7	Controller	Redireciona para /login com a mensagem “Cadastro realizado com sucesso!”.

Fonte: Elaborado pelo autor (2026).

O login segue um caminho parecido, mas quem conduz é o Spring Security. O filtro de login recebe e-mail e senha, o UsuarioDetailsService busca o usuário pelo e-mail normalizado e devolve o hash e o perfil, e o PasswordEncoder compara a senha digitada com o hash. Se tudo confere, a sessão é criada e gravada na coleção sessoes.

2.3 DTOS E MAPPER

Os DTOs (Data Transfer Objects) são records Java que carregam só os dados de cada operação. O UsuarioRequestDTO tem nome, e-mail e senha e é usado no cadastro. O UsuarioAtualizacaoRequestDTO tem nome, e-mail e perfil e é usado na edição feita pelo administrador. O UsuarioResponseDTO tem id, nome, e-mail e perfil e é o que chega às páginas.

Essa separação tem um motivo de segurança. A entidade tem os campos `senhaHash` e `perfil`. Se o formulário fosse ligado direto a ela, alguém poderia enviar o perfil ADMIN no cadastro ou ver o hash numa página. Como o DTO de cadastro não tem o campo `perfil` e o DTO de resposta não tem o hash, isso não acontece. Além disso, as regras de validação de cada formulário ficam no seu DTO, e o mesmo campo pode ter regras diferentes em operações diferentes.

O `UsuarioMapper` concentra a conversão entre entidade e DTO. Controllers e serviços não montam objetos campo a campo, e se a entidade ganhar um atributo novo, a mudança fica em um lugar só.

3 INTEGRAÇÃO COM O MONGODB ATLAS

O MongoDB Atlas guarda os dois tipos de dado do sistema: os usuários e as sessões. Os usuários são gravados pelo Spring Data MongoDB, e as sessões pela biblioteca mongodb-spring-session.

3.1 CONFIGURAÇÃO E VARIÁVEL MONGODB_URI

O endereço de conexão nunca aparece no código. O arquivo `application.properties` só faz referência à variável de ambiente `MONGODB_URI`, e cada máquina que executa o sistema define essa variável com a URI do seu cluster. O arquivo versionado no GitHub não tem credenciais e fica assim:

```
spring.application.name=login-seguro
spring.mongodb.uri=${MONGODB_URI}
spring.mongodb.database=login_seguro
spring.data.mongodb.auto-index-creation=true
app.visual.nome=Login Seguro
app.visual.tema=padrao
```

A URI é copiada do próprio Atlas, na opção de conexão para aplicações, e tem o formato abaixo (valores fictícios):

```
mongodb+srv://USUARIO:SENHA@SEU_CLUSTER.mongodb.net/?retryWrites=true
&w=majority
```

No IntelliJ, a variável é cadastrada nas configurações de execução da classe `LoginSeguroApplication`. No terminal do Windows, ela pode ser definida na sessão do PowerShell antes de rodar o Maven Wrapper. Se o usuário ou a senha do banco tiverem caracteres especiais, eles precisam ser codificados para uso na URI. O README do repositório descreve os dois procedimentos.

3.2 BANCO, COLEÇÕES E ÍNDICE

O banco se chama login_seguro e tem as coleções do quadro 4.

Quadro 4 – Coleções do banco login_seguro

Coleção	Conteúdo	Responsável no código
usuarios	Um documento por usuário, com _id, nome, email, senhaHash e perfil.	UsuarioEntity e IUsuarioRepository
sessoes	Uma entrada por sessão, com os atributos da sessão e o usuário autenticado. Expira após 30 minutos sem atividade.	SessaoConfig (@EnableMongoHttpSession)

Fonte: Elaborado pelo autor (2026).

O campo email da entidade tem a anotação `@Indexed(unique = true)`, e a propriedade `spring.data.mongodb.auto-index-creation=true` faz o Spring criar esse índice único quando a aplicação sobe. A verificação no serviço mostra uma mensagem clara ao usuário, e o índice garante a regra no próprio banco.

3.3 CONEXÃO SEGURA, USUÁRIO DO BANCO E LIBERAÇÃO DO IP

A URI do tipo `mongodb+srv` fornecida pelo Atlas usa TLS por padrão. Os dados trafegam criptografados entre a aplicação e o cluster, e a validação do certificado continua habilitada.

A aplicação deve se conectar com um usuário de banco próprio, com permissão de leitura e escrita apenas no banco login_seguro, e não com a conta de administração do Atlas. Assim, se essa senha vazar, o acesso fica limitado a esse banco.

O Atlas também só aceita conexões de endereços cadastrados na lista de acesso de rede (Network Access). O IP da máquina que executa a aplicação precisa estar nessa lista. Quando o IP muda, por exemplo ao trocar de rede, a conexão falha até o novo endereço ser adicionado.

3.4 PRIMEIRO ADMINISTRADOR

Todo cadastro feito pela aplicação recebe o perfil USER, e só um ADMIN altera perfis. Por isso, o primeiro administrador é preparado direto no Atlas:

- a) cadastrar a conta normalmente pela página /cadastro;

- b) sair da conta;
- c) no Atlas, abrir o banco login_seguro e a coleção usuarios;
- d) localizar o documento pelo e-mail cadastrado e alterar o campo perfil de USER para ADMIN;
- e) entrar de novo no sistema.

A partir daí, os perfis das outras contas são alterados pela área administrativa. O sistema não cria administradores automaticamente nem usa variáveis de ambiente para isso.

4 PERSONALIZAÇÃO E ADAPTAÇÃO AO PFC

No meu projeto, a ideia de reaproveitar o sistema no PFC guiou a organização desde o começo. Esta seção mostra o que precisa mudar quando o sistema for adaptado a outro tema.

4.1 NOME DO SISTEMA E TEMA

O nome e o tema ficam no `application.properties`, nas propriedades com prefixo `app.visual`, lidas pela classe `TemaConfig`:

```
app.visual.nome=Login Seguro  
app.visual.tema=verde
```

O valor de `app.visual.nome` aparece no título da aba, no cabeçalho e no rodapé de todas as páginas. O valor de `app.visual.tema` escolhe qual arquivo da pasta `static/css/temas` é carregado. Hoje existem `padrao.css` e `verde.css`. O `TemaControllerAdvice` entrega essas propriedades a todas as páginas com o nome `visual`, então nenhum controller precisa repassá-las. Depois de trocar os valores, basta reiniciar a aplicação, sem mudar nenhuma classe Java.

O nome do tema só aceita letras minúsculas, números e hífen. A `TemaConfig` valida isso na inicialização, e um valor fora desse padrão impede a aplicação de subir. O sistema não confere se o arquivo CSS existe: se o nome estiver no formato certo mas o arquivo faltar, as páginas abrem sem as cores do tema.

4.2 CRIAR UM NOVO TEMA

Para criar um tema, copio um dos arquivos da pasta `static/css/temas` com outro nome, por exemplo `azul-escuro.css`, troco as três variáveis de cor e configuro `app.visual.tema=azul-escuro`, sem a extensão:

```
:root {  
  --cor-primaria: #17613a;  
  --cor-fundo: #f1f7f2;  
  --cor-texto: #203527;  
}
```

O `estilo.css` usa essas variáveis nos botões, links, fundo e texto, então o novo arquivo muda a aparência de todas as páginas de uma vez.

4.3 FRAGMENTOS COMPARTILHADOS

O arquivo `templates/fragmentos/layout.html` define três fragmentos: `head(titulo)`, que carrega as folhas de estilo; `cabecalho`, com a marca, a imagem e o menu; e `rodape`. Cada página chama esses fragmentos com `th:replace`. Trocar o logotipo, acrescentar um item no menu ou mudar o rodapé é uma alteração nesse único arquivo. A imagem do cabeçalho fica em `static/imagens` e é referenciada no fragmento `cabecalho`. Os links do menu usam `sec:authorize`, então um item novo pode aparecer só para os perfis certos.

4.4 NOVAS FUNCIONALIDADES E REGRAS DE ACESSO

Uma funcionalidade nova segue o caminho do quadro 3. Na prática, os passos são:

- a) criar a página em `templates`, usando os fragmentos de `layout.html`;
- b) criar ou ampliar um controller com a rota, sem colocar regra de negócio nele;
- c) declarar o método na interface do pacote `service` e implementar as regras em `service.impl`;
- d) se houver dados novos, criar a entidade, o repositório, os DTOs e o mapper correspondentes;
- e) liberar a rota na `SecurityConfig`, no bloco `authorizeHttpRequests`, indicando quais perfis podem acessá-la;
- f) se a página tiver link no menu, acrescentá-lo no fragmento `cabecalho` com `sec:authorize`.

Se o PFC precisar de outros perfis, eles entram na enumeração `PerfilEnum` e nas regras da `SecurityConfig`. Como os nomes atuais são genéricos, também dá para manter os três níveis e mudar só o texto exibido nas páginas.

5 REQUISITOS E REGRAS DE NEGÓCIO

Segundo Sommerville (2011, p. 59), os requisitos funcionais descrevem os serviços que o sistema oferece e como ele se comporta diante de entradas e situações específicas. Os requisitos não funcionais foram classificados, como propõe o mesmo autor (Sommerville, 2011, p. 61), em requisitos de produto, organizacionais e externos. Esta seção apresenta os atores, os requisitos com a situação de cada um e as regras de negócio.

5.1 ATORES E PERMISSÕES

Quadro 5 – Atores do sistema

Ator	Descrição
Visitante	Pessoa sem sessão iniciada. Cria conta e entra no sistema.
USER	Perfil de quem se cadastra. Acessa a página inicial e sai do sistema.
MANAGER	Perfil intermediário. Tem o acesso do USER e entra na área de gerenciamento.
ADMIN	Responsável pelo sistema. Tem o acesso do MANAGER e lista, edita e exclui usuários.
Sistema	Ações automáticas: controle de acesso às rotas, expiração das sessões e aplicação do tema.

Fonte: Elaborado pelo autor (2026).

Separar os perfis segue o princípio apresentado por Sommerville (2011, p. 267) de dar a cada usuário só o acesso de que ele precisa, o que diminui o estrago de uma conta comprometida. O servidor confere as permissões em toda requisição. O cabeçalho só esconde os links que o perfil não pode usar, e quem digita a URL direto continua barrado. O quadro 6 mostra o que cada perfil acessa.

Quadro 6 – Permissões por perfil

Funcionalidade (rotas)	Visitante	USER	MANAGER	ADMIN
Cadastrar conta e entrar (/cadastro, /login)	Sim	—	—	—
Página inicial e sair (/inicio, /logout)	Não	Sim	Sim	Sim
Área de gerenciamento (/gerenciamento)	Não	Não	Sim	Sim
Administração e lista de usuários (/admin, /admin/usuarios)	Não	Não	Não	Sim
Editar e excluir usuários (/admin/usuarios/{id}/editar e /excluir)	Não	Não	Não	Sim

Fonte: Elaborado pelo autor (2026).

Nota: o traço indica que a ação não se aplica a quem já está autenticado.

Hoje a área de gerenciamento só mostra uma mensagem de acesso. Ela existe para receber, no PFC, as funções do perfil intermediário.

5.2 REQUISITOS FUNCIONAIS

Quadro 7 – Resumo dos requisitos funcionais

ID	Requisito	Situação
RF01	Cadastrar usuário	Atendido
RF02	Autenticar usuário	Atendido
RF03	Encerrar sessão	Atendido
RF04	Controlar o acesso às páginas por perfil	Atendido
RF05	Exibir a página inicial do usuário autenticado	Atendido. A página mostra o e-mail. Nome, perfil e data de cadastro ficam para uma futura página “Meu perfil”.
RF06	Listar usuários	Atendido. Somente ADMIN, sem busca e sem paginação.
RF07	Editar usuário (nome, e-mail e perfil)	Atendido
RF08	Excluir usuário	Atendido
RF09	Aplicar o nome e o tema visual configurados	Atendido

Fonte: Elaborado pelo autor (2026).

5.3 REQUISITOS NÃO FUNCIONAIS

Quadro 8 – Resumo dos requisitos não funcionais

ID	Requisito	Situação
RNF01	Senhas só como hash BCrypt	Atendido
RNF02	Autenticação e autorização pelo Spring Security	Atendido (verificação manual)
RNF03	Proteção CSRF nos formulários	Atendido
RNF04	Cookie de sessão protegido	Parcial (atributo Secure não configurado)
RNF05	Conexão segura com o Atlas e URI fora do código	Atendido
RNF06	Escape dos dados exibidos (XSS)	Atendido
RNF07	Código organizado em camadas	Atendido
RNF08	Interface separada da lógica, com temas em CSS	Atendido
RNF09	Perfis e regras de acesso em um único lugar	Atendido
RNF10	Mensagens de validação em português	Atendido
RNF11	Uso em telas a partir de 360 pixels	Não verificado
RNF12	Java 21, Spring Boot 4.1.1, Spring Security e Thymeleaf	Atendido
RNF13	Usuários e sessões no MongoDB Atlas	Atendido
RNF14	Repositório no GitHub com GitFlow	Atendido
RNF15	README com instruções de execução	Atendido
RNF16	Documentação em PDF conforme a ABNT	Atendido

Fonte: Elaborado pelo autor (2026).

5.4 REGRAS DE NEGÓCIO

As regras de negócio valem para todo o sistema e são conferidas no servidor, mesmo as que a interface também verifica.

Quadro 9 – Regras de negócio: cadastro, login e sessão

ID	Regra	Descrição
RN01	E-mail único	Cada e-mail pertence a uma única conta. O e-mail é guardado e comparado sem espaços nas pontas e em letras minúsculas, e o banco tem índice único nesse campo.
RN02	Dados válidos	O nome deve ter de 2 a 100 caracteres e o e-mail deve ter formato válido, com até 254 caracteres. A validação acontece no servidor, mesmo quando o navegador também valida.
RN03	Tamanho da senha	A senha deve ter no mínimo 8 caracteres e no máximo 72 bytes em UTF-8, o limite do BCrypt. Nesta versão não há campo de confirmação nem exigência de letra maiúscula, número ou símbolo.
RN04	Sigilo da senha	A senha é guardada só como hash BCrypt e nunca é exibida nas páginas nem devolvida pelo serviço.
RN05	Perfil inicial	Todo cadastro público recebe o perfil USER. Só um ADMIN altera perfis pela aplicação. O primeiro administrador é preparado alterando o campo perfil do seu documento no Atlas (seção 3.4).
RN06	Mensagem genérica de login	Uma tentativa de login malsucedida sempre mostra “E-mail ou senha inválidos.”, sem revelar se o e-mail existe.
RN07	Sessão	A sessão é gravada na coleção sessoes do Atlas e expira após 30 minutos sem atividade. O cookie SESSION usa HttpOnly e SameSite=Lax.
RN08	Destino após o login	Todos os perfis são levados a /inicio. O cabeçalho mostra só as opções permitidas ao perfil.
RN09	Encerramento da sessão	O logout é feito por POST para /logout com token CSRF. A sessão é invalidada, os cookies SESSION e JSESSIONID são apagados e o login exibe a mensagem de saída.

Fonte: Elaborado pelo autor (2026).

Quadro 10 – Regras de negócio: acesso, administração e tema

ID	Regra	Descrição
RN10	Acesso por rota	São públicas /, /login, /cadastro, /css/**, /imagens/** e /error (a rota / só redireciona para /inicio). /gerenciamento exige MANAGER ou ADMIN. /admin e /admin/** exigem ADMIN. Qualquer outra rota, incluindo /inicio, exige autenticação.
RN11	Acesso negado	Quem abre uma rota protegida sem sessão vai para o login. Quem tem sessão, mas não tem o perfil exigido, recebe a página de acesso negado com status HTTP 403.
RN12	Proteção do administrador	O administrador não pode excluir a própria conta nem alterar o próprio e-mail ou perfil pela aplicação. Essas mudanças exigem outra conta ADMIN.
RN13	Efeito imediato	Excluir uma conta ou alterar o e-mail ou o perfil marca as sessões do usuário como expiradas. Na próxima requisição ele volta ao login com o aviso de sessão encerrada.
RN14	Tema	O nome do tema só pode ter letras minúsculas, números e hífen, validado na inicialização. O sistema carrega o arquivo de mesmo nome em static/css/temas e não confere se ele existe.
RN15	Listagem de usuários	Só ADMIN lista usuários. A lista traz todos os cadastrados em ordem alfabética de nome, sem busca e sem paginação.
RN16	Edição de usuário	O ADMIN pode alterar nome, e-mail e perfil (USER, MANAGER ou ADMIN), respeitando RN01, RN02 e RN12.
RN17	Exclusão de usuário	A exclusão pede confirmação no navegador e remove a conta definitivamente. Não existe desativação nem reativação.

Fonte: Elaborado pelo autor (2026).

6 CONSIDERAÇÕES FINAIS

O sistema entregue cumpre o que a atividade pede: cadastro, login e logout, senhas com hash BCrypt, validação no servidor, três perfis com controle de acesso por rota, e usuários e sessões guardados no MongoDB Atlas. A divisão em camadas e a separação entre interface e regras permitem adaptar o projeto ao PFC mexendo na configuração, no CSS e nos fragmentos.

Para mim, foi útil escrever a primeira versão deste documento antes do código e depois compará-la com o que implementei. Algumas ideias do planejamento ficaram de fora, como a confirmação de senha e a desativação de contas. Outras surgiram durante o desenvolvimento, como a edição de usuários pelo administrador e o encerramento das sessões de quem é alterado ou excluído.

Esta versão tem limitações que pretendo tratar nas próximas etapas:

- a) a senha exige só o mínimo de 8 caracteres, sem campo de confirmação nem exigência de letra maiúscula, número e símbolo;
- b) não existe uma página “Meu perfil” com nome, perfil e data de cadastro, e a entidade ainda não guarda a data de cadastro;
- c) a lista de usuários não tem busca nem paginação, o que vai pesar quando houver muitos registros;
- d) a exclusão é definitiva, sem opção de desativar uma conta e reativá-la depois;
- e) o sistema não confere se o arquivo CSS do tema configurado existe;
- f) as permissões foram testadas só manualmente, e faltam testes automatizados com MockMvc;
- g) o atributo Secure do cookie não foi configurado, e a aplicação não foi testada publicada com HTTPS;
- h) a área de gerenciamento do perfil MANAGER ainda não tem funções próprias;
- i) o uso em telas estreitas não foi testado.

REFERÊNCIAS

MONGODB. **MongoDB Atlas documentation**. 2026. Disponível em: <https://www.mongodb.com/docs/atlas/>. Acesso em: 8 out. 2026.

SILVA, Alessandro Aparecido da. **Guia de apoio**: como descrever requisitos funcionais para um projeto de software. Mogi das Cruzes: Universidade de Mogi das Cruzes, 2026.

SOMMERVILLE, Ian. **Engenharia de software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011.

SPRING. **Spring Boot reference documentation**. 2026. Disponível em: <https://docs.spring.io/spring-boot/index.html>. Acesso em: 8 out. 2026.

SPRING. **Spring Data MongoDB reference documentation**. 2026. Disponível em: <https://docs.spring.io/spring-data/mongodb/reference/index.html>. Acesso em: 8 out. 2026.

SPRING. **Spring Security reference**. 2026. Disponível em: <https://docs.spring.io/spring-security/reference/index.html>. Acesso em: 8 out. 2026.

THYMELEAF. **Tutorial**: using Thymeleaf. 2026. Disponível em: <https://www.thymeleaf.org/doc/tutorials/3.1/usingthymeleaf.html>. Acesso em: 8 out. 2026.